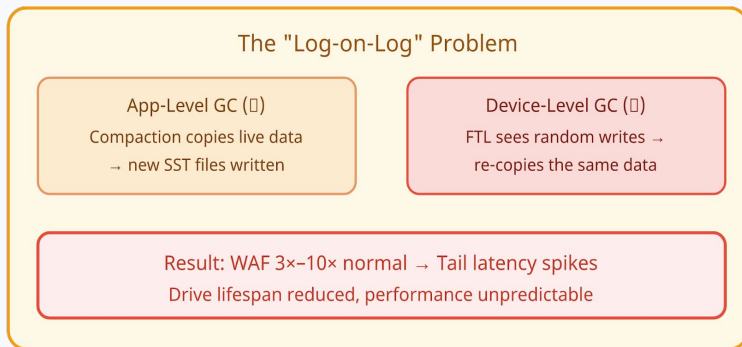
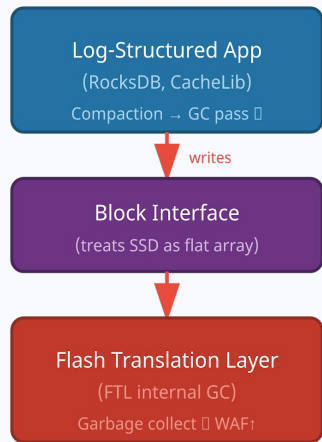


Evaluation of ZNS and FDP

Howard Huang · CS2640

The Problem: The Semantic Gap



Legacy Block Interface

Treats SSDs as uniform, updatable arrays — ignores flash physics.

NAND Reality

Flash requires out-of-place writes & coarse-grained erase blocks.

Log-on-Log Problem

App GC + FTL GC independently copy the same data → WAF 3-10×.

Consequence

Tail latency spikes, degraded throughput, reduced drive lifespan.

Bridging the Gap: Two Paradigms

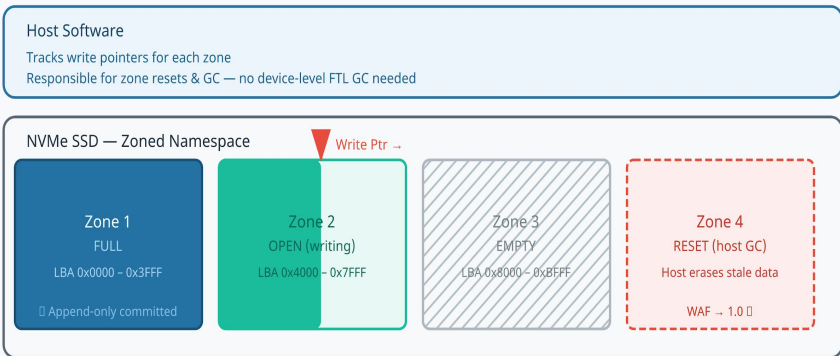
Zoned Namespaces (ZNS)

- Strict append-only zones
- Host manages Write Pointers
- ✓ WAF approaches 1.0 (no FTL GC)
- ✗ Full software rewrite required

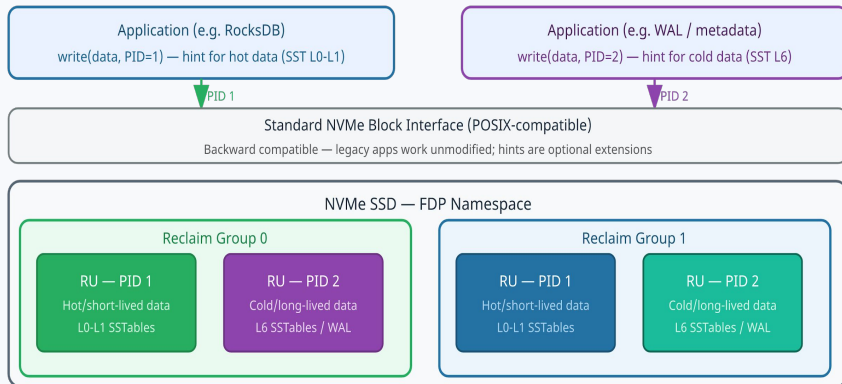
Flexible Data Placement (FDP)

- Maintains standard block interface
- Host passes Placement ID hints
- ✓ ~90% WAF reduction, zero friction
- ✓ Backward-compatible, POSIX-native

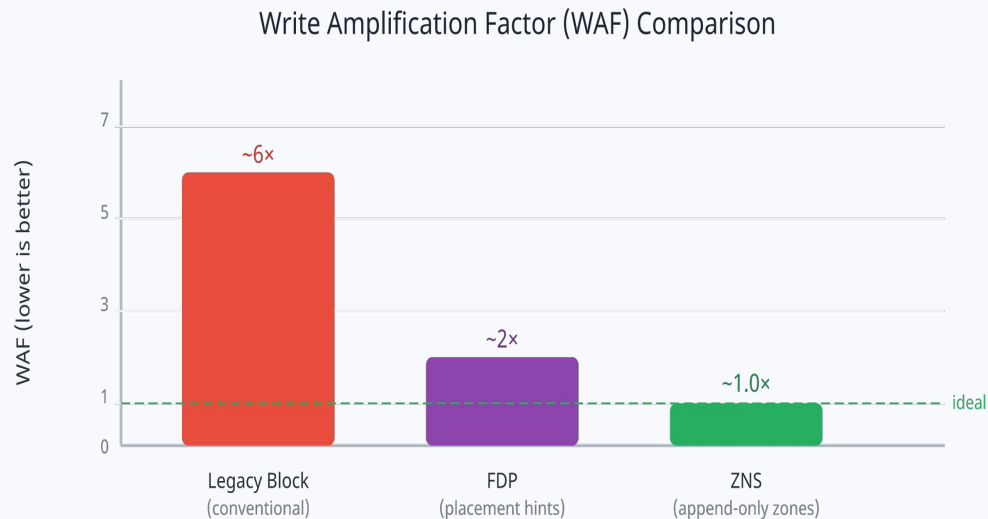
Zoned Namespaces (ZNS) Architecture



Flexible Data Placement (FDP) Architecture



Why It Matters: Write Amplification



3x–10x Legacy WAF

~2x FDP WAF

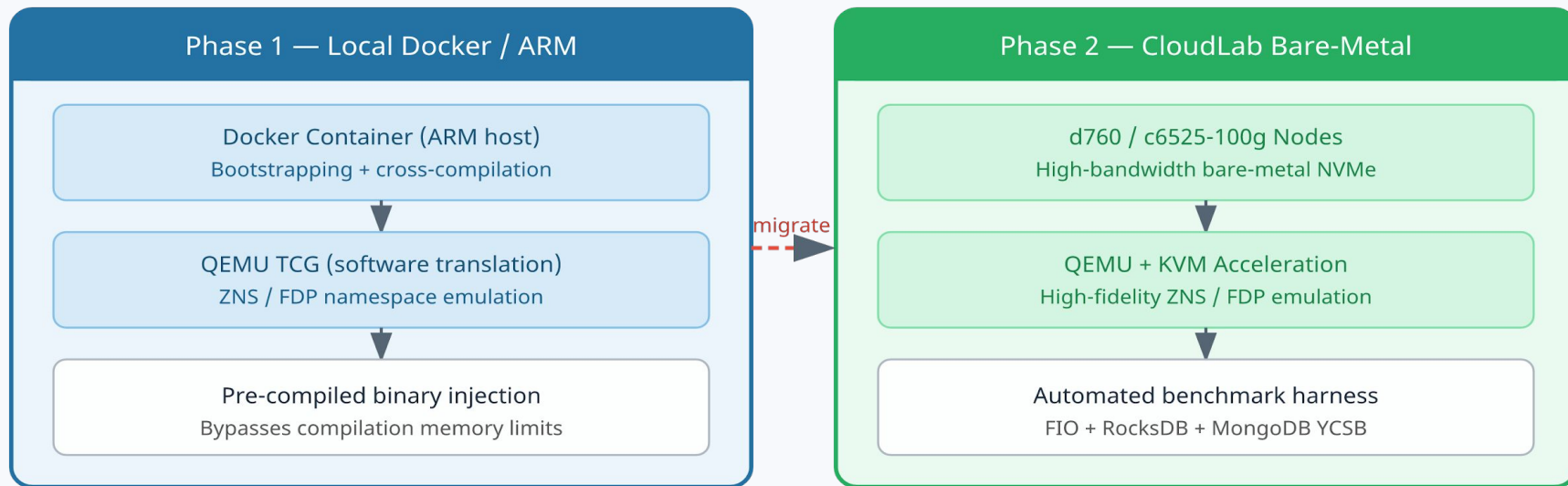
1.0x ZNS WAF

*Reducing WAF from 6x to 1x means 6x more write bandwidth
and dramatically longer SSD lifespan.*

Emulation Methodology

Challenge: Physical Gen 5 SSDs with ZNS/FDP firmware are extremely scarce.

Dual-Phase Emulation Framework



CloudLab nodes: d760 & c6525-100g — high-bandwidth NVMe with KVM acceleration for high-fidelity emulation

Benchmark Workloads

01

FIO Synthetic Traces

Simulates CacheLib CDN workloads using `io_uring_cmd` NVMe passthrough. Validates device-level WAF approaches 1.0 on ZNS.

02

RocksDB — db_bench

Key-value store benchmarks: fillseq, fillrandom, readrandom, overwrite. Stresses LSM-tree compaction and GC behavior.

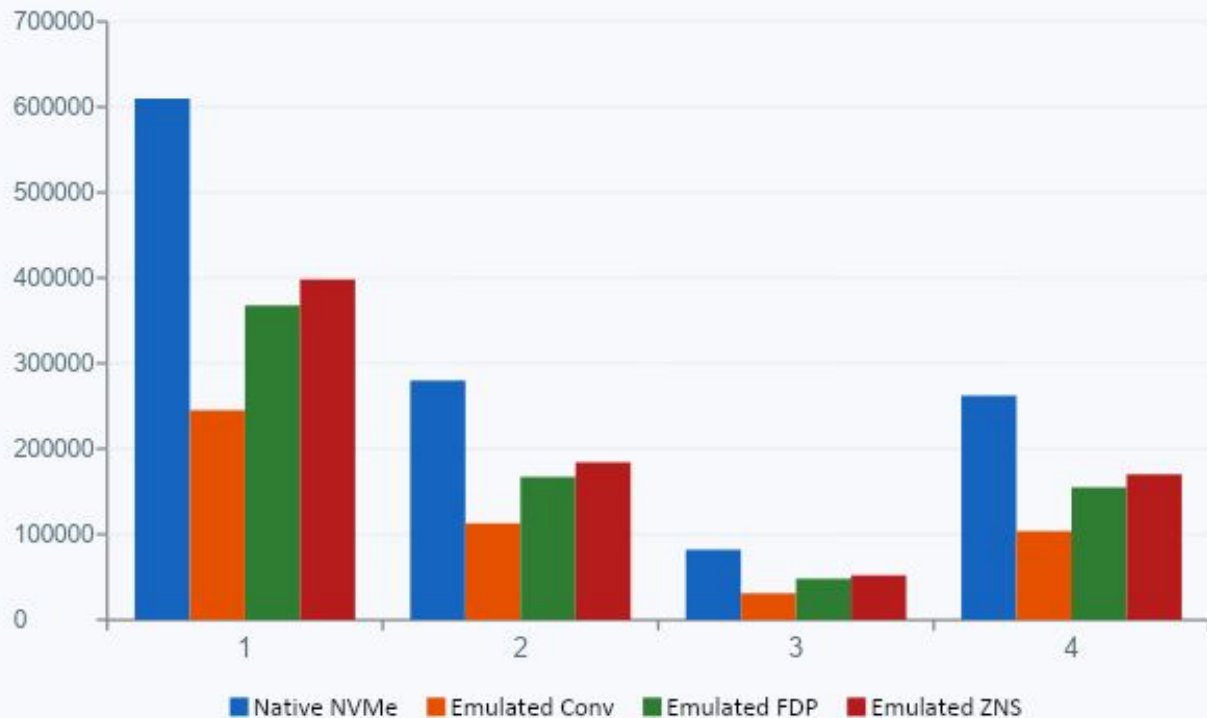
03

MongoDB — YCSB

1,000,000 document records across 4 YCSB workloads (A/B/C/F). Evaluates filesystem-layer overhead under mixed read/write ratios.

Results: RocksDB Throughput

RocksDB — Ops/sec by Workload & Device



Native

610k ops/sec peak on fillseq — the baseline ceiling.

ZNS

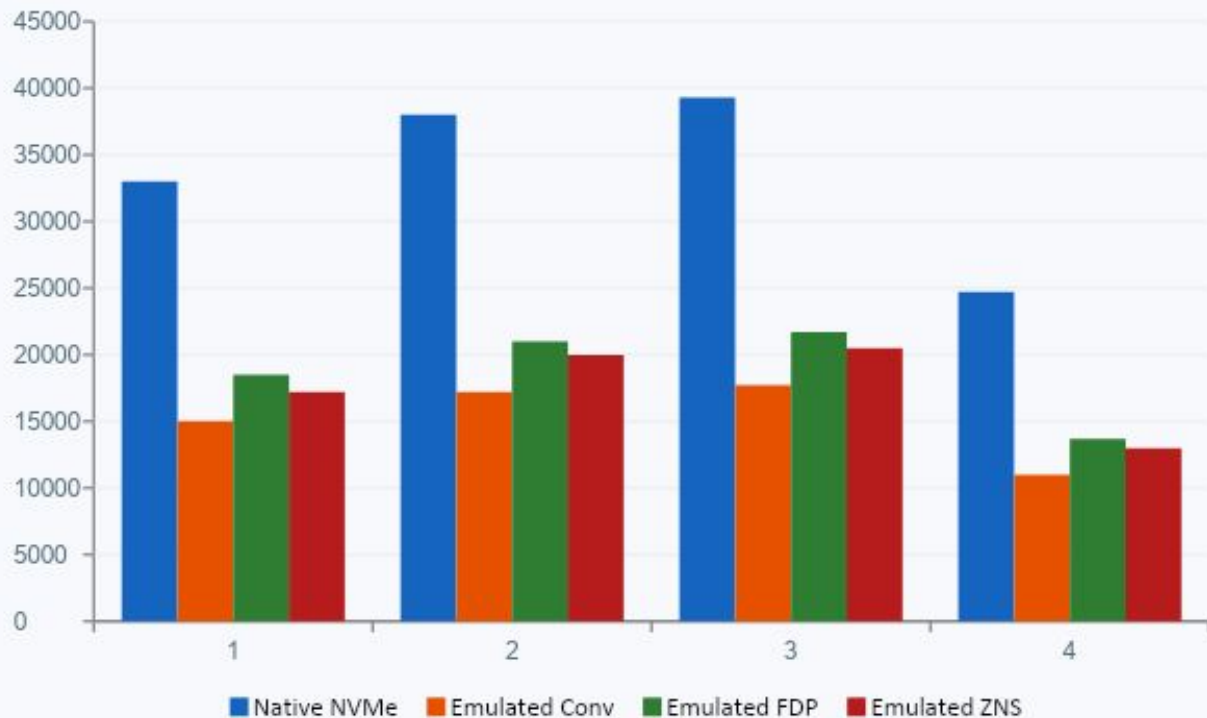
+62% vs Emulated Conv — zero internal GC amplification.

FDP

Bridges the gap without append-only constraints.

Results: MongoDB YCSB

MongoDB YCSB — Ops/sec by Workload & Device



Filesystem Overhead

ZNS requires f2fs (host-side translation). Under QEMU software emulation this incurs a CPU penalty.

FDP advantage

POSIX ops pass through natively with transparent PID injection — no filesystem overhead.

Key Insight

FDP pulls ahead of ZNS in filesystem-heavy workloads while maintaining simpler integration.

Learnings & Limitations



The Virtualization Tax is Real

QEMU introduces non-trivial I/O wait times that distort raw μs latency metrics vs bare silicon. Throughput ceilings reflect CPU emulation, not flash cell limits.



Hypervisor Maturity Gap

QEMU 6.2 on CloudLab lacks native FDP argument support, requiring complex proxying workarounds. Newer hypervisors and firmware are needed for clean benchmarks.



The Clear Trade-Off

ZNS offers absolute WAF control (1.0 $\times$) but demands complete software rearchitecture. FDP delivers $\sim 90\%$ of the benefit with practically zero integration friction.

Future Work

01 Hardware Acquisition

Deploy the test harness directly onto **physical PCIe Gen 5 SSDs** with native ZNS/FDP firmware.

- Capture unadulterated latency distributions
- Eliminate the QEMU virtualization tax
- Validate emulation model accuracy

02 eBPF Auto-Placement Module

Kernel-level eBPF module that intercepts I/O calls and

autonomously injects FDP

Placement Identifiers

based on file descriptor.

- Trace file descriptors at the kernel level
- Route I/O threads to correct FDP PIDs
- **Zero application-level code modifications**

Goal: Legacy applications utilize next-gen SSDs with zero source code modifications

Key Takeaways

The semantic gap is real — FTL GC + app GC creates WAF 3–10×, degrading perf & lifespan.

ZNS = maximum control — WAF → 1.0× but requires complete software rearchitecture.

FDP = maximum pragmatism — ~90% WAF reduction with full backward compatibility.

eBPF unlocks the future — Zero-modification legacy app integration via kernel-level PID injection.